

SkillBuilderPro Architecture Documentation

Overview

This document details all architectural improvements implemented in the SkillBuilderPro Web API. These changes transform the codebase from a basic CRUD API into an **enterprise-grade, loosely coupled, testable application** following SOLID principles and Clean Architecture patterns.

What Changed & Why

BEFORE (Basic CRUD)

```
csharp

// Old approach: Direct database access in controller
[HttpGet("{id}")]
public async Task<Drill> GetDrill(int id)
{
    return await _context.Drills.FirstOrDefaultAsync(d => d.Id == id);
}

// Problems:
// - Tight coupling (controller knows about DbContext)
// - Hard to test (can't mock database)
// - Mixed concerns (data access + business logic)
// - Not reusable (logic locked in controller)
```

AFTER (Enterprise Architecture)

```
csharp
```

```
// New approach: Dependency injection + Repository pattern
[HttpGet("{id}")]
public async Task<Drill> GetDrill(int id)
{
    var drill = await _drillService.GetDrillByIdAsync(id);
    if (drill == null)
        return NotFound();
    return Ok(drill);
}

// Benefits:
// - Loose coupling (controller doesn't know about DbContext)
// - Easy to test (inject mock repository)
// - Separation of concerns (controller only handles HTTP)
// - Reusable logic (service used by multiple controllers)
```

New Components & Their Roles

1. IRepository<T> Interface (Generic Repository Pattern)

File: SkillBuilderPro.API/Repositories/IRepository.cs

What it is: A generic interface that defines a contract for all data access operations. It abstracts away the details of HOW data is accessed from WHAT data operations are performed.

What it does:

- `GetByIdAsync(int id)` — Fetch single entity by primary key (O(1) complexity)
- `GetAllAsync()` — Fetch all entities (O(n) complexity)
- `FindAsync(Expression<Func<T, bool>> predicate)` — Fetch entities matching a condition (O(n) complexity)
- `AddAsync(T entity)` — Add new entity to database
- `UpdateAsync(T entity)` — Modify existing entity

- `DeleteAsync(T entity)` — Remove entity from database
- `SaveAsync()` — Commit all changes to database

Architectural Pattern: Repository Pattern

- **Purpose:** Mediate between domain layer (services) and data mapping layer (Entity Framework)
- **Benefit:** Decouples business logic from database implementation
- **Why it matters:** If you switch from SQL Server to PostgreSQL, only repository changes — controllers and services don't

Interview talking point:

"I implemented the Repository Pattern to abstract data access. This means the service layer doesn't know if we're using SQL Server, PostgreSQL, or even a REST API for data — we just swap the repository implementation. It's testable too — in unit tests, we can inject a mock repository that returns hardcoded data instead of hitting the database."

2. DrillRepository
UserRepository
ProgressRepository (Repository Implementations)

Files:

- `SkillBuilderPro.API/Repositories/DrillRepository.cs`
- `SkillBuilderPro.API/Repositories/UserRepository.cs`
- `SkillBuilderPro.API/Repositories/ProgressRepository.cs`

What they are: Concrete implementations of `IRepository<T>` for each domain entity. Each repository is responsible for all database operations for its specific entity type.

What they do:

DrillRepository:

- Queries the Drills table using Entity Framework Core

- Filters drills by sport using LINQ expressions
- Adds, updates, deletes drills
- Example: `FindAsync(d => d.Sport == "basketball")` returns all basketball drills

UserRepository:

- Queries the Users table
- Finds users by username for authentication
- Manages user CRUD operations

ProgressRepository:

- Queries the Progress table (athlete performance history)
- Tracks which drills each athlete completed
- Logs completion dates and repetition counts

Architectural Pattern: Repository Pattern (Entity-Specific Implementation)

- **Purpose:** One repository per entity type for single responsibility
- **Benefit:** Each repository is focused, testable, and maintainable
- **Complexity tracking:**
 - `GetByIdAsync()` — $O(1)$ via primary key index
 - `FindAsync()` — $O(n)$ via table scan
 - `AddAsync()` — $O(1)$ in-memory, $O(\log n)$ when saving (B-tree insert)

Interview talking point:

"Each repository implements the same interface but handles different entities. This is the Single Responsibility Principle in action — DrillRepository only knows about drills, UserRepository only about users. It makes unit testing trivial because I can create a mock repository that returns test data."

Code example — Finding drills by sport (from DrillRepository):

csharp

```
public async Task<IEnumerable<Drill>> FindAsync(Expression<Func<Drill, bool>> predicate)
{
    return await _context.Drills.Where(predicate).ToListAsync();
}

// Used by DrillService:
var basketballDrills = await _drillRepository.FindAsync(d => d.Sport == "basketball");
// This translates to: SELECT * FROM Drills WHERE Sport = 'basketball'
```

3. IDrillService
IAuthService
IProgressService (Service Interfaces)

Files:

- SkillBuilderPro.API/Services/IDrillService.cs
- SkillBuilderPro.API/Services/IAuthService.cs
- SkillBuilderPro.API/Services/IProgressService.cs

What they are: Interfaces that define business logic contracts. These are the "API" of the business logic layer — services implement these to provide specific functionality.

What they do:

IDrillService:

- GetAllDrillsAsync() — Fetch all 60 drills
- GetDrillByIdAsync(int id) — Fetch specific drill
- GetDrillsBySportAsync(string sport) — Filter drills by sport
- CreateDrillAsync(Drill drill) — Add new drill to library
- UpdateDrillAsync(Drill drill) — Modify drill
- DeleteDrillAsync(int id) — Remove drill

IAuthService:

- `AuthenticateAsync(username, password)` — Verify user credentials (returns user object if valid)
- `RegisterAsync(username, email, password, role)` — Create new user account
- `GetAllUsersAsync()` — Fetch all users
- `GetUserByIdAsync(int id)` — Fetch specific user

IProgressService:

- `LogDrillCompletionAsync(userId, drillId, reps)` — Record when athlete completes a drill
- `GetUserProgressAsync(userId)` — Get all drills completed by an athlete
- `GetAllProgressAsync()` — Get all progress entries (for analytics)
- `GetDrillCompletionCountAsync(userId)` — Count how many drills athlete has done

Architectural Pattern: Service Layer + Dependency Injection

- **Purpose:** Encapsulate business logic separate from HTTP concerns (controllers)
- **Benefit:** Logic is reusable, testable, and can be called from multiple consumers (WinForms, MAUI, other APIs)
- **Why it matters:** Business logic lives in services, not controllers. Controllers only handle HTTP

Interview talking point:

"I separated business logic from HTTP controllers using a service layer. This means the authentication logic lives in AuthService, not in the controller. If I wanted to add a gRPC endpoint or a background job that uses the same logic, I just inject AuthService into it. Everything is reusable and testable."

4. DrillService
AuthService
ProgressService (Service Implementations)

Files:

- SkillBuilderPro.API/Services/DrillService.cs
- SkillBuilderPro.API/Services/AuthService.cs
- SkillBuilderPro.API/Services/ProgressService.cs

What they are: Concrete implementations of service interfaces. These contain the actual business logic.

What they do:

DrillService:

```
csharp
```

```
public async Task<IEnumerable<Drill>> GetDrillsBySportAsync(string sport)
{
    // Business logic: normalize sport name (case-insensitive)
    return await _drillRepository.FindAsync(d =>
        d.Sport.ToLower() == sport.ToLower()
    );
}

public async Task<bool> DeleteDrillAsync(int id)
{
    // Business logic:
    // 1. Check if drill exists
    // 2. Delete it
    // 3. Commit changes
    // 4. Return success/failure
    var drill = await _drillRepository.GetByIdAsync(id);
    if (drill == null) return false;

    await _drillRepository.DeleteAsync(drill);
    await _drillRepository.SaveAsync();
    return true;
}
```

AuthService:

csharp


```
public async Task<User> AuthenticateAsync(string username, string password)
{
    // Business logic:
    // 1. Find user by username
    // 2. Verify password matches
    // 3. Return user object if credentials valid
    var users = await _userRepository.FindAsync(u => u.Username == username);
    var user = users.FirstOrDefault();

    if (user == null) return null; // User doesn't exist
    if (user.PasswordHash != password) return null; // Wrong password

    return user; // Credentials valid
}

// Note: In production, passwords would be hashed with bcrypt/Argon2
```

ProgressService:

csharp

```
public async Task<Progress> LogDrillCompletionAsync(int userId, int drillId,
{
    // Business logic: Create a new progress record
    var progress = new Progress
    {
        UserId = userId,
        DrillId = drillId,
        CompletedDate = DateTime.UtcNow,
        RepetitionsCompleted = reps
    };

    await _progressRepository.AddAsync(progress);
    await _progressRepository.SaveAsync();
    return progress;
}

public async Task<int> GetDrillCompletionCountAsync(int userId)
{
    // Business logic: Count drills completed by athlete
    var progress = await _progressRepository.FindAsync(p => p.UserId == userId);
    return progress.Count(); // Simple analytics
}
```

Architectural Pattern: Service Layer (Business Logic)

- **Purpose:** Contain all business rules, validation, and orchestration
- **Benefit:** Reusable across controllers, UI clients, background jobs
- **Dependency flow:** Service → Repository → Database

Interview talking point:

"Services encapsulate business logic. For example, deleting a drill isn't just 'DELETE FROM Drills' — we first check if it exists, then delete it, then commit. That logic lives in the service layer, not scattered across controllers. If I need to call this from WinForms or a background job, I just inject the service and call the same method."

5. Updated Program.cs (Dependency Injection Configuration)

File: SkillBuilderPro.API/Program.cs

What it is: The application startup and configuration file. Contains the Dependency Injection container setup.

What it does:

```
csharp

// Register repositories - each entity gets its own repository
builder.Services.AddScoped<IRepository<Drill>, DrillRepository>();
builder.Services.AddScoped<IRepository<User>, UserRepository>();
builder.Services.AddScoped<IRepository<Progress>, ProgressRepository>();

// Register services - inject repositories into services
builder.Services.AddScoped<IDrillService, DrillService>();
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<IProgressService, ProgressService>();
```

What `AddScoped` means:

- "Scoped" = one instance per HTTP request
- Each HTTP request gets fresh instances of services/repositories
- When request completes, instances are disposed
- Ensures data isolation and thread safety

Dependency Injection Chain:

```
HTTP Request arrives
    ↓
DrillsController needs IDrillService
    ↓
DI container creates DrillService
    ↓
DrillService constructor needs IRepository<Drill>
    ↓
DI container creates DrillRepository
    ↓
DrillRepository constructor needs AppDbContext
    ↓
DI container creates DbContext
    ↓
Everything wired up, request handled
    ↓
Request completes, all instances disposed
```

Architectural Pattern: Dependency Injection (IoC Container)

- **Purpose:** Automatic wiring of dependencies, enables loose coupling
- **Benefit:** Controllers don't know HOW to create services — they just request them
- **Testing benefit:** Swap real repository with mock repository for unit tests

Interview talking point:

"I use the built-in .NET dependency injection container in Program.cs. Controllers declare what they need through constructor injection, and the DI container automatically provides it. This is 'Inversion of Control' — the container controls object creation, not the controller. For testing, I can swap the real DrillRepository with a mock that returns test data."

6. Updated DrillsController (Dependency Injection in Controller)

File: SkillBuilderPro.API/Controllers/DrillsController.cs

What changed:

BEFORE:

```
csharp

[ApiController]
[Route("api/[controller]")]
public class DrillsController : ControllerBase
{
    private readonly AppDbContext _context;

    public DrillsController(AppDbContext context)
    {
        _context = context;
    }

    [HttpGet]
    public async Task<IActionResult> GetDrills()
    {
        // Direct database access – tight coupling
        var drills = await _context.Drills.ToListAsync();
        return Ok(drills);
    }
}

// Problems:
// - Controller knows about DbContext (tight coupling)
// - Can't test without real database
// - Business logic scattered in controller
```

AFTER:

csharp

```
[ApiController]
[Route("api/[controller]")]
public class DrillsController : ControllerBase
{
    private readonly IDrillService _drillService;

    // Inject service, not DbContext
    public DrillsController(IDrillService drillService)
    {
        _drillService = drillService;
    }

    [HttpGet]
    [ProducesResponseType(StatusCodes.Status200OK)]
    public async Task<ActionResult<IEnumerable<Drill>>> GetDrills()
    {
        var drills = await _drillService.GetAllDrillsAsync();
        return Ok(drills);
    }

    [HttpGet("{id}")]
    [ProducesResponseType(StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<ActionResult<Drill>> GetDrill(int id)
    {
        var drill = await _drillService.GetDrillByIdAsync(id);
        if (drill == null)
            return NotFound(new { message = $"Drill with ID {id} not found." });
        return Ok(drill);
    }

    /// <summary>
    /// Gets drills filtered by sport.
}
```

```

    /// </summary>
    /// <remarks>
    /// Complexity: O(n) where n is number of drills
    /// Can be optimized with Sport index on database
    /// </remarks>
    [HttpGet("sport/{sport}")]
    public async Task<ActionResult<IEnumerable<Drill>>> GetDrillsBySport(string sport)
    {
        var drills = await _drillService.GetDrillsBySportAsync(sport);
        return Ok(drills);
    }
}

// Benefits:
// - Controller only handles HTTP (single responsibility)
// - Loose coupling to service interface
// - Easy to test (inject mock service)
// - XML comments document each method
// - ProducesResponseType documents possible responses

```

Key improvements:

1. **Constructor injection:** `IDrillService` injected, not `DbContext`
2. **Separation of concerns:** Controller handles HTTP, service handles logic
3. **XML documentation:** Each method documented with summary, remarks, complexity
4. **Response type documentation:** `ProducesResponseType` tells API consumers what responses to expect
5. **Proper error handling:** Returns 404 with meaningful message if drill not found

Architectural Pattern: Separation of Concerns

- **Purpose:** Controller only cares about HTTP, service only cares about business logic
- **Benefit:** Each class has one job, easy to test each layer independently

Interview talking point:

"The controller no longer talks directly to the database. It delegates to the service layer. This is the 'Dependency Inversion Principle' — high-level modules (controllers) depend on abstractions (IDrillService), not low-level modules (Entity Framework). If I want to change the database, I only change the repository. Controllers don't care."

How It All Works Together

Request Flow — GET /api/drills/5

1. HTTP GET /api/drills/5 arrives at DrillsController
2. DrillsController.GetDrill(5) called
↓
3. Calls _drillService.GetDrillByIdAsync(5)
↓
4. DrillService.GetDrillByIdAsync(5) called
↓
5. Calls _drillRepository.GetByIdAsync(5)
↓
6. DrillRepository.GetByIdAsync(5) called
↓
7. Executes: _context.Drills.FirstOrDefault(d => d.Id == 5)
↓
8. Entity Framework translates to SQL:
SELECT * FROM Drills WHERE Id = 5
↓
9. SQL Server returns drill record
↓
10. Entity Framework materializes Drill object
↓
11. Drill object returned up the chain:
Repository → Service → Controller
↓
12. Controller returns HTTP 200 OK with Drill JSON
↓
13. Client receives response

Complexity analysis:

- Step 8-9: $O(1)$ because Id is primary key (indexed)
 - Step 7 (EF LINQ query): $O(1)$
 - Total: $O(1)$ constant time lookup
-

SOLID Principles Demonstrated

S — Single Responsibility Principle

- **DrillRepository:** Only handles Drill data access
- **DrillService:** Only handles Drill business logic
- **DrillsController:** Only handles HTTP requests/responses
- Each class has one reason to change

O — Open/Closed Principle

- Repositories are **open for extension** (can add new methods)
- **Closed for modification** (existing code doesn't change)
- Can add new repository without touching existing ones

L — Liskov Substitution Principle

- Any `IRepository<T>` can replace another `IRepository<T>`
- Controller doesn't care if it's real repository or mock
- Enables testing without breaking production code

I — Interface Segregation Principle

- `IRepository<T>` interface small and focused
- `IDrillService` interface only includes drill-related methods
- Clients depend only on methods they use

D — Dependency Inversion Principle

- **High-level:** DrillsController (business logic)
- **Low-level:** DrillRepository (data access)

- **Both depend on:** Interfaces (IDrillService, IRepository<T>)
 - High-level doesn't depend on low-level directly
-

Testing Benefits

Unit Test Example — No Database Required

```
csharp
```

```

[TestClass]
public class DrillServiceTests
{
    private Mock<IRepository<Drill>> _mockRepository;
    private DrillService _drillService;

    [TestInitialize]
    public void Setup()
    {
        // Create mock repository that returns fake data
        _mockRepository = new Mock<IRepository<Drill>>();
        _drillService = new DrillService(_mockRepository.Object);
    }

    [TestMethod]
    public async Task GetDrillsBySport_WithBasketball_ReturnBasketballDrills()
    {
        // Arrange: Setup mock to return test data
        var expectedDrills = new List<Drill>
        {
            new Drill { Id = 1, Name = "Crossover Dribble", Sport = "basketball" },
            new Drill { Id = 2, Name = "Bounce Pass", Sport = "basketball" }
        };

        _mockRepository
            .Setup(r => r.FindAsync(It.IsAny<Expression<Func<Drill, bool>>>())
            .ReturnsAsync(expectedDrills);

        // Act: Call service method
        var result = await _drillService.GetDrillsBySportAsync("basketball");

        // Assert: Verify results
        Assert.AreEqual(2, result.Count());
    }
}

```

```

        Assert.IsTrue(result.All(d => d.Sport == "basketball"));

        // Verify repository was called once
        _mockRepository.Verify(r => r.FindAsync(It.IsAny<Expression<Func<Drill>()>>()), Times.Once);
    }
}

// This test:
// - Runs in milliseconds (no database)
// - Doesn't require SQL Server running
// - Returns predictable results (no data variability)
// - Isolated from other tests (mock data only)

```

Benefits:

-  Tests run fast (no I/O)
-  Tests are deterministic (same data every time)
-  Can test edge cases easily (inject empty list, null, exceptions)
-  Decoupled from database schema changes

Deployment Impact

Database can change (Repository abstraction)

```

Old: SQL Server only
New: Can switch to PostgreSQL without changing controllers/services
     Just swap repository implementation

```

Performance can be optimized (Service layer)

```

Old: Must change controller to add caching logic
New: Add caching in service layer, controller unchanged

```

New features can be added (Loose coupling)

Old: Adding MAUI app means copying/pasting controller logic

New: MAUI injects same services as API, reuses all business logic

Interview Talking Points Summary

1. Repository Pattern:

"I abstracted data access using repositories. The service layer depends on `IRepository<T>`, not EF Core directly. This means if we change databases, only the repository changes—controllers and services stay the same."

2. Service Layer:

"Business logic lives in services, not controllers. This makes it reusable. The same `DrillService` can be used by REST API, WinForms desktop app, MAUI mobile app, or background jobs. Each just injects the service."

3. Dependency Injection:

"I use constructor injection throughout. The DI container automatically wires up dependencies. In tests, I swap the real repository with a mock, so tests run fast without hitting the database."

4. Separation of Concerns:

"Each class has one job: controller handles HTTP, service handles business logic, repository handles database. This makes code easier to test, modify, and understand."

5. SOLID Principles:

"I followed SOLID throughout. Single Responsibility (each class has one job), Open/Closed (open for extension, closed for modification), and Dependency Inversion (high-level code depends on abstractions, not low-level details)."

6. Testability:

"Every layer is testable independently. Controllers can be tested by mocking services. Services can be tested by mocking repositories. This means I can test business logic without a database running."

File Structure Summary

```
SkillBuilderPro.API/
├─ Program.cs                ← DI container setup
├─ Controllers/
│   ├─ DrillsController.cs   ← HTTP endpoints, uses IDrillService
│   ├─ UsersController.cs   ← HTTP endpoints, uses IAuthService
│   └─ ProgressController.cs ← HTTP endpoints, uses
IPressService
├─ Services/
│   ├─ IDrillService.cs      ← Interface (contract)
│   ├─ DrillService.cs       ← Implementation (business logic)
│   ├─ IAuthService.cs       ← Interface
│   ├─ AuthService.cs        ← Implementation
│   ├─ IPressService.cs      ← Interface
│   └─ PressService.cs       ← Implementation
└─ Repositories/
    ├─ IRepository.cs        ← Generic interface (contract)
    ├─ DrillRepository.cs     ← Implementation (data access)
    ├─ UserRepository.cs     ← Implementation (data access)
    └─ ProgressRepository.cs  ← Implementation (data access)
```

Dependency chain:


```
Controller
    ↓ (depends on)
Service Interface
    ↓ (implemented by)
Service Implementation
    ↓ (depends on)
Repository Interface
    ↓ (implemented by)
Repository Implementation
    ↓ (uses)
DbContext
    ↓ (maps to)
SQL Server Database
```

Each layer is swappable, testable, and independent.

Next Steps

1.  **Done:** Repository Pattern + Service Layer + DI
2. **Next:** Add Exception Handling Middleware (Priority 4)
3. **Then:** Add XML Comments to all public methods (Priority 3)
4. **Optional:** Add unit tests (Priority 5)

This architecture is **production-grade** and shows enterprise-level thinking. Perfect for interviews. 